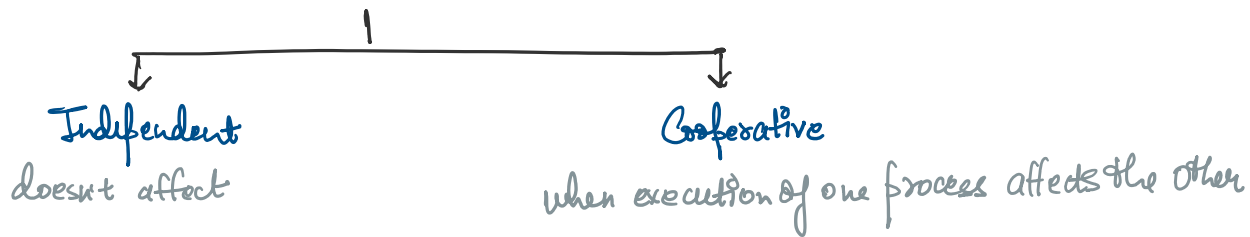


* Interprocess Communication

mechanism provided by the OS that allows processes to communicate with each other

Used for exchanging useful information between numerous threads in one or more processes (or program)

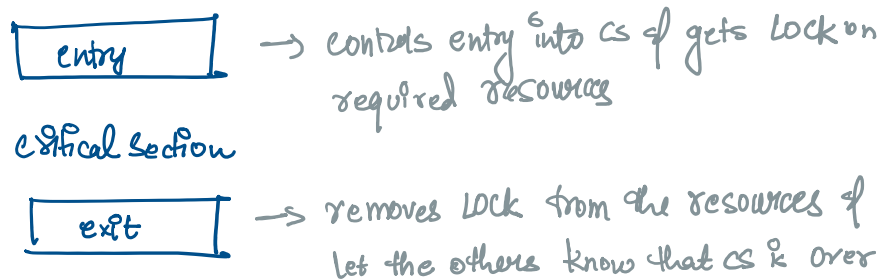
* Process Synchronization introduced to handle problems that arise while multiple process executions.



* Process Synchronisation is mainly needed in a multi-process system when multiple processes are running together, if more than one process try to gain access to the same shared resource or any data at the same time.

* Critical Section

- code segment where the shared variables can be accessed.
- Only one process can execute in its critical section at a time while others wait

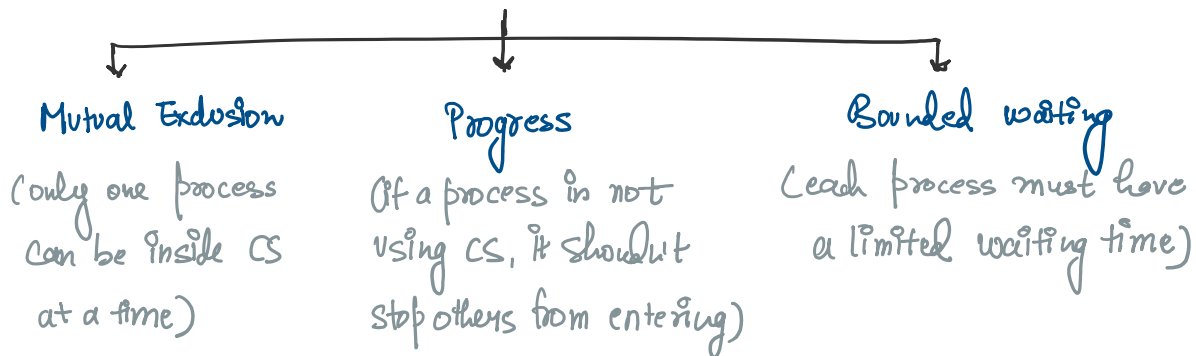


* Race Condition

- At a time when more than one process is either executing the same code or accessing the same memory or any shared variable.
- There is a possibility that the output or the value of shared variable is wrong

- There is a possibility that the output or the value of shared variable is wrong so for that purpose all the processes are doing race to say their output is correct.
- This is Race condition that may occur inside the critical section.
- When the result of multiple thread execution differs according to the order in which threads execute.

Solution to Critical Section Problem



★ Lock Variables

- Simplest synchronization mechanism for processes
- Implemented in user mode; no support from OS
- busy waiting solution
- can be used for more than two processes
- Lock = 0 (empty CS), Lock = 1 (occupied CS)

Entry Section - while (lock != 0);

Lock = 1;

// critical section

Exit Section - Lock = 0;

★ TSL Instructions (hardware based)

Test-and-Set Lock (TSL) is synchronization mechanism in OS to manage access to shared resources in a concurrent environment

- Tests the value of a memory location
- Sets it to specified value (usually 1)

Load Lock, R0

CMP R0, #0

Test and set

Load Lock, R0

Store #1, lock

CMP R0, #0

b. Sets it to specified value (usually 1)

(ii) lock starts at 0

(iii) Process 1 check the lock;

If lock = 0, it changes lock = 1 & enters CS

If lock = 1, it keeps waiting (busy-waiting) until lock = 0

(iv) Process 2 has to wait until process 1 is done

(v) When Process 1 finishes, it sets lock = 0 & process 2 can proceed.

```
CMP R0, #0
```

```
JNZ step1
```

```
Store #1, lock
```

Store #1, lock

```
CMP R0, #0
```

```
JNZ step1
```

★ Peterson's Solution (software based)

- Whenever a process is executing in CS, then the other process only executes the rest of the code, & vice versa can happen.

- Preserves all three conditions

→ Two shared variable

boolean flag[i] : Initialised false
ready to enter CS

int turn[i] : Turn to enter CS

→ Disadvantages: Busy Waiting
limited to 2 processes

```
do {
```

```
    flag[i] = TRUE;
```

```
    turn[i] = i;
```

```
    while (flag[j] && turn == j);
```

```
    // critical section
```

```
    flag[i] = FALSE;
```

```
    // remainder section
```

```
} while (TRUE);
```

★ Busy Waiting & Sleep Waiting

- Synchronization technique where the process or task continuously use the processor & waits for the condition to be satisfied.

- Sleep Waiting : a process where the task or process doesn't consume the processor when waiting for the condition to be satisfied.

- Need of Busy Waiting : for achieving Mutual Exclusion

- Limitations : consumes more power

keeps CPU busy at all times

low priority process gets executed

low priority process gets executed
system remains idle

★ Sleep & Wake up call

If the critical section is not empty, the process will go to sleep. It will be waked up by the other process which is currently executing inside the critical section so that P_i can use critical section.

➔ Producer Consumer Problem

- Have the 2 processes
- One process writes something (Producer)
- Other process reads that (Consumer)

→ $N=100$ // max slots in buffer
 $count=0$ // items in buffer

```
void producer(void)
{
    int item;
    while (True)
    {
        item = produce_item(); // producer produces an item
        if (count == N) // if buffer is full the producer will sleep
            sleep();
        insert_item(item) // item is inserted
        count = count + 1
        if (count == 1) // producer will wake up consumer if there is one
            wake-up(consumer); // item in the buffer)
    }
}
```

→

```
void customer(void)
{
    int item;
    while (True)
    {
        if (count == 0) // consumer will sleep if the buffer is empty
```

```

{
    if (count == 0) // consumer will sleep if the buffer is empty
        sleep();
    item = remove - item();
    count = count - 1;
    if (count == N - 1) // if at least one available slot in buffer
        wakeup - up (producer); // consumer will wake up the producer
    consume - item (item);
}
}

```

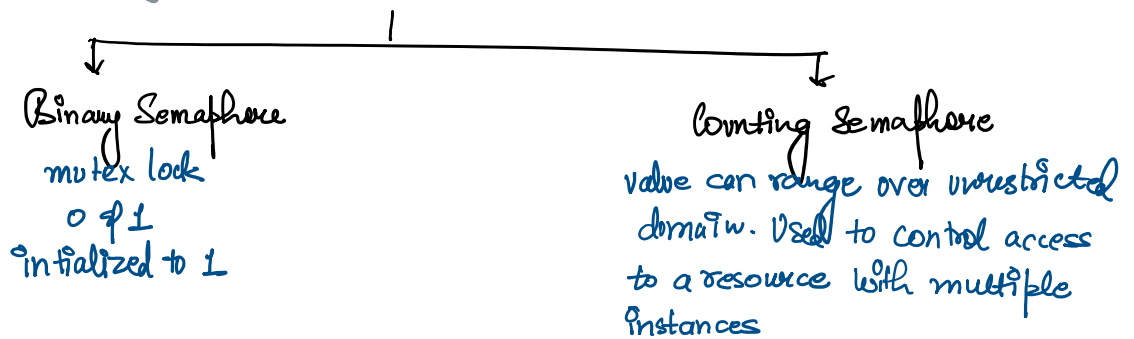
* Semaphores

An integer variable that is shared b/w threads. Used to solve critical section problems & to achieve process synchronization in the multiprocessing environment

(i) wait(P) decrements value of semaphore (down, sleep)

(ii) Signal(V) increments value of semaphore (up, wake)

if $s = 0$; any process that performs P will be blocked until V



```

→ P(Semaphores) {
    while (s == 0);
    s = s - 1;
}

```

```

V(Semaphores) {
    s = s + 1;
}

```

```

// entry section
P(s);
// critical section
V(s);
// remainder section

```

Advantages : Mutual Exclusion strictly followed
No resource wastage
Machine independent

Disadvantages : Complexity



Machine independent

Disadvantages: Complicated
Impractical
May lead to priority inversion

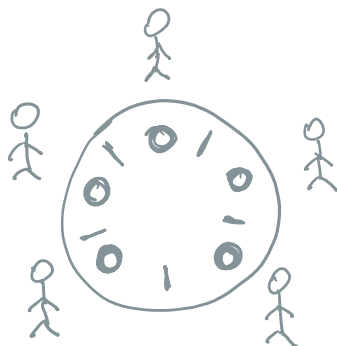


Limitations: Priority Inversion
Deadlock
OS has to keep track of all calls
Require Busy waiting

★ Monitors

- Higher-level Synchronization construct that simplifies process synchronization by providing a high level abstraction for data access & synchronization.
- Implemented as programming lang. constructs typically in object-oriented languages & provide mutual exclusion, condition variables, data encapsulation in a single construct.
- essentially a module that encapsulates a shared resource & provides access to that resource through a set of procedures.
- procedures ensure that only one process can access the shared resource at a time
- Advantages: eliminate the need for complex synchronization primitives (semaphores, locks)
Simple-high level abstraction

★ Dining Philosophers



Solution- Semaphores

★ Readers Writer Problem

Readers -  etc
Writer - 

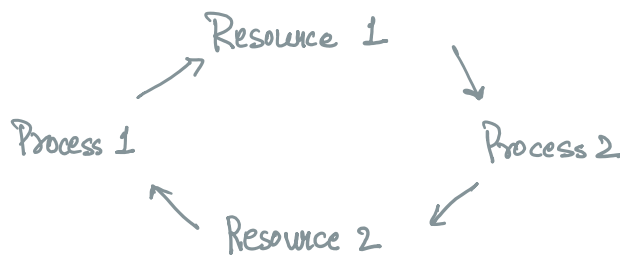
Solutions-

- Readers Preference
- Writers Preference

* Deadlocks

Situation where two or more processes are unable to proceed because each is waiting for the other to release resources.

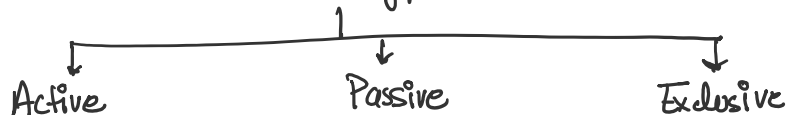
Condition where 2 or more processes sharing the same resource are effectively preventing each other from accessing the resource



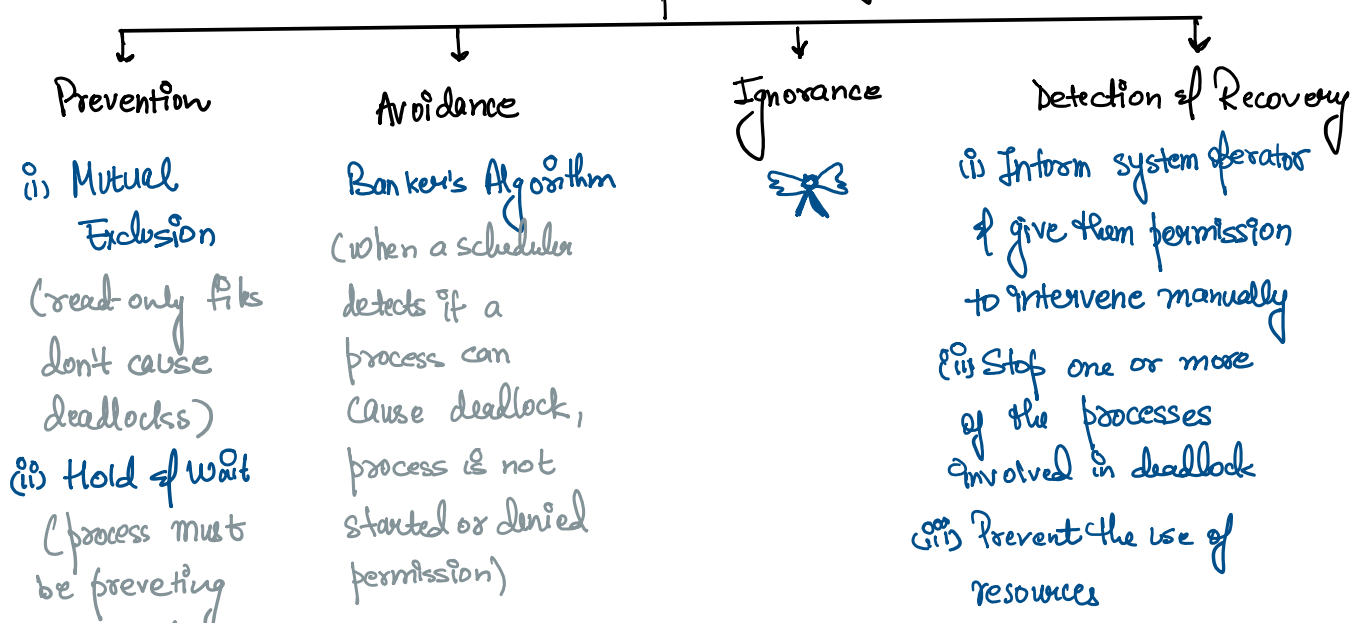
→ Operations - Request
Use
Release } typical ways to use a resource

→ Conditions for Deadlock - Mutual Exclusion
Hold & Wait
No Preemption
Circular Wait

Resource Types



Methods for Deadlock Handling



(process must
be preventing
from holding one
or more
resources)

(permission)

Can prevent the use of
resources

(ii) No Preemption

(i) Circular Wait